



UNIVERSIDADE FEDERAL DO CEARÁ
Campus Quixadá
Curso: Redes de Computadores
Disciplina: Redes

Aula 3 – Camada de Aplicação (Introdução)

Prof. Rafael Braga

Capítulo 2: Roteiro

- 2.1 Princípios de aplicações de rede
- 2.2 A Web e o HTTP
- 2.3 Transferência de arquivo: FTP
- 2.4 Correio Eletrônico na Internet
- 2.5 DNS: o serviço de diretório da Internet
- 2.6 Aplicações P2P
- 2.7 Programação de *sockets* com TCP
- 2.8 Programação de *sockets* com UDP
- 2.9 Construindo um servidor Web simples

Capítulo 2: Camada de Aplicação

Metas do capítulo:

- aspectos conceituais e de implementação de protocolos de aplicação em redes
 - modelos de serviço da camada de transporte
 - paradigma cliente servidor
 - paradigma *peer-to-peer*
- ▶ aprenda sobre protocolos através do estudo de protocolos populares da camada de aplicação:
 - HTTP
 - FTP
 - SMTP/ POP3/ IMAP
 - DNS
- ▶ a programação de aplicações de rede
 - programação usando a API de *sockets*

Algumas aplicações de rede

- Correio eletrônico
- A Web
- Mensagem instantânea
- Login em computador remoto como Telnet e SSH
- Compartilhamento de arquivos P2P
- Jogos multiusuários em rede
- Videoclipes armazenados
- Telefonia por IP
- Videoconferência em tempo real
- Computação paralela massiva
- ?
- ?
- ?

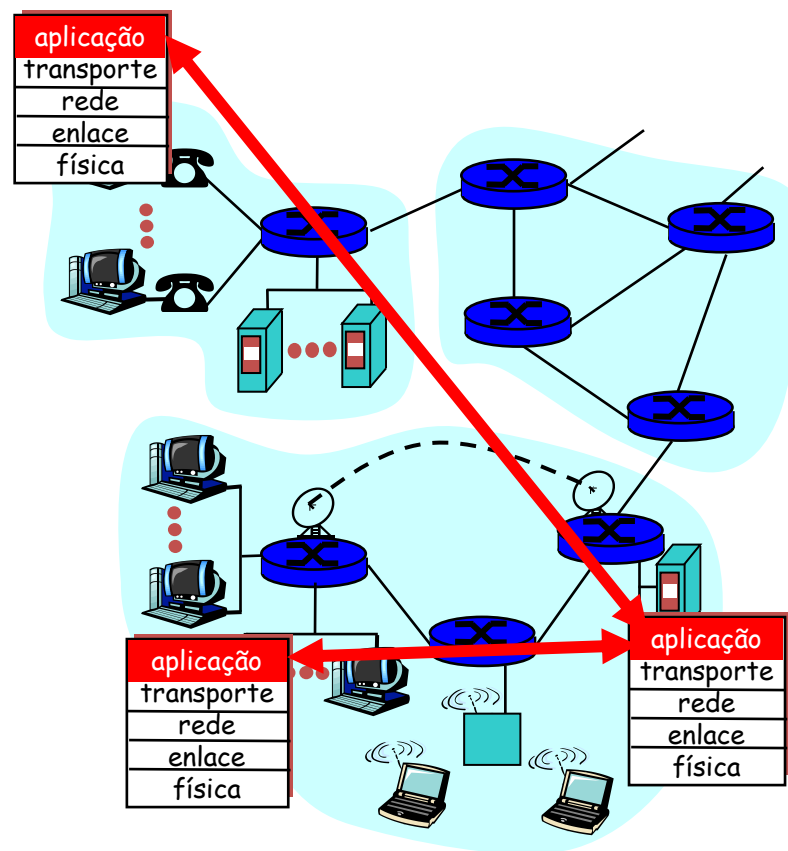
Criando uma aplicação de rede

Programas que

- Executam em diferentes sistemas finais
- Comunicam-se através da rede
- p.ex., Web: servidor Web se comunica com o navegador

Programas não relacionados ao núcleo da rede

- Dispositivos do núcleo da rede não executam aplicações de usuários
- Aplicações nos sistemas finais permite rápido desenvolvimento e disseminação



Capítulo 2: Roteiro

- 2.1 Princípios de aplicações de rede
- 2.2 A Web e o HTTP
- 2.3 Transferência de arquivo: FTP
- 2.4 Correio Eletrônico na Internet
- 2.5 DNS: o serviço de diretório da Internet
- 2.6 Aplicações P2P
- 2.7 Programação de *sockets* com TCP
- 2.8 Programação de *sockets* com UDP
- 2.9 Construindo um servidor Web simples

Arquiteturas das aplicações de rede

- Cliente-servidor
- Peer-to-peer (P2P)
- Híbrida cliente-servidor/P2P

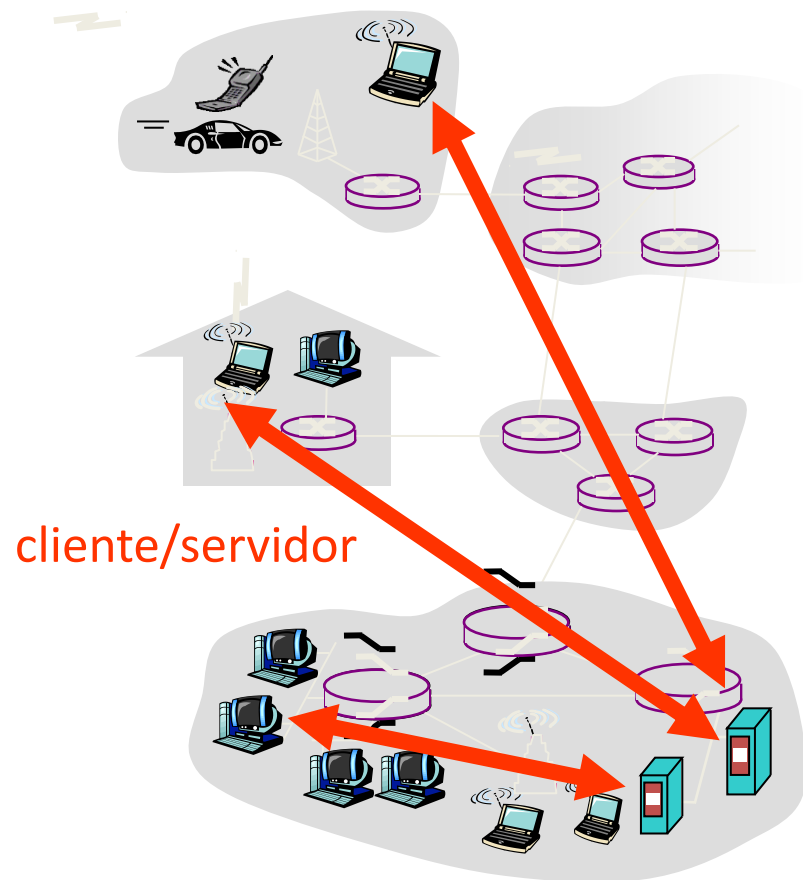
Arquitetura cliente-servidor

Servidor:

- ❑ Sempre ligado
- ❑ Endereço IP permanente
- ❑ Escalabilidade com *server farms*

Cliente:

- ❑ Comunica-se com o servidor
- ❑ Pode estar conectado intermitentemente
- ❑ Pode ter endereços IP dinâmicos
- ❑ Não se comunica diretamente com outros clientes

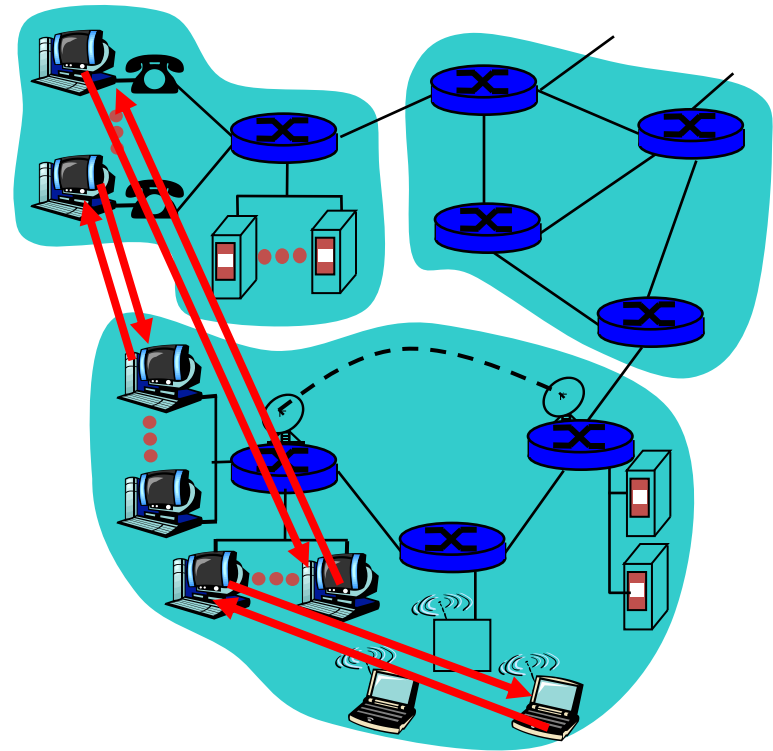


Arquitetura P2P pura

- ▶ Não há servidor sempre ligado
- ▶ Sistemas finais arbitrários se comunicam diretamente
- ▶ Pares estão conectados intermitentemente e mudam endereços IP

Altamente escalável

Porém, difícil de gerenciar



Híbrida cliente-servidor e P2P

Skype

- Aplicação P2P de voz sobre IP
- servidor centralizado: localização do endereço do parceiro remoto
- conexão cliente-cliente: direta (sem ter que passar por um servidor)

Mensagem instantânea

- Conversa entre usuários P2P
- Localização e detecção de presença centralizadas:
 - Usuários registram o seu endereço IP junto ao servidor central quando ficam online
 - Usuários consultam o servidor central para encontrar endereços IP dos contatos

Comunicação entre Processos

Processo: programa que executa num sistema final

- ▶ processos no mesmo sistema final se comunicam usando **comunicação interprocessos** definida pelo sistema operacional (SO)
- ▶ processos em sistemas finais distintos se comunicam trocando **mensagens** através da rede

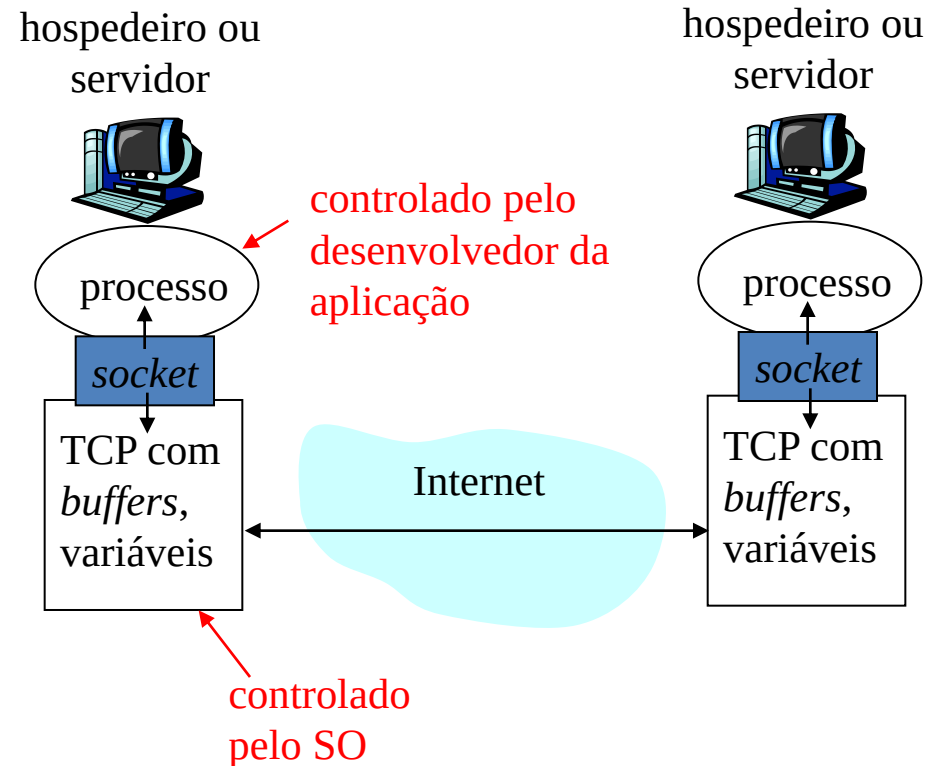
Processo cliente: processo que inicia a comunicação

Processo servidor: processo que espera ser contatado

- ❑ Nota: aplicações com arquiteturas P2P possuem processos clientes e processos servidores

Sockets

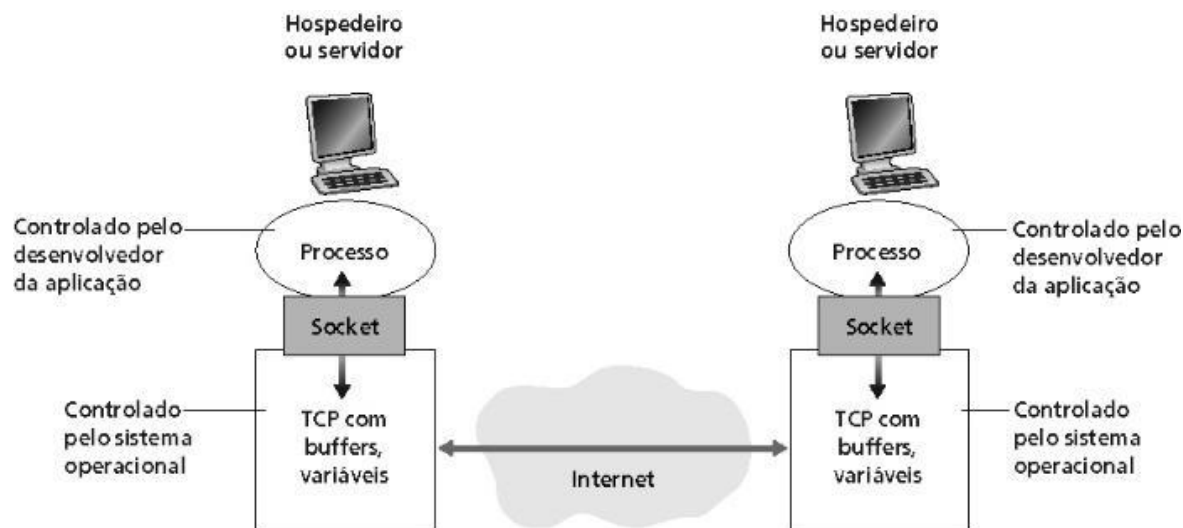
- Os processos enviam/recebem mensagens para/dos seus *sockets*
- *Um socket* é análogo a uma porta
 - Processo transmissor envia a mensagem através da porta
 - O processo transmissor assume a existência da infra-estrutura de transporte no outro lado da porta que faz com que a mensagem chegue ao *socket* do processo receptor



- ❑ API: (1) escolha do protocolo de transporte; (2) habilidade para fixar alguns parâmetros (*mais sobre isto posteriormente*)

Endereçamento de processos

- Para que um processo receba mensagens, ele deve possuir um **identificador**
- Cada hospedeiro possui um **endereço IP** único de 32 bits
- **P**: o endereço IP do hospedeiro no qual o processo está sendo executado é suficiente para identificar o processo?



Endereçamento de processos

- O identificador inclui tanto o endereço IP quanto os **números das portas** associadas com o processo no hospedeiro .
 - Exemplo de números de portas:
 - Servidor HTTP: 80
 - Servidor de Correio: 25
 - Para enviar uma msg HTTP para o servidor Web gaia.cs.umass.edu
 - **Endereço IP:** 128.119.245.12
 - **Número da porta:** 80
 - **Mais sobre isto posteriormente.**
- ▶ Para que um processo receba mensagens, ele deve possuir um **identificador**
 - ▶ Cada hospedeiro possui um **endereço IP** único de 32 bits
 - ▶ **P:** o endereço IP do hospedeiro no qual o processo está sendo executado é suficiente para identificar o processo?
 - ▶ **Resposta:** Não, muitos processos podem estar executando no mesmo hospedeiro

Os protocolos da camada de aplicação definem

► Tipos de mensagens trocadas:

- ex. mensagens de requisição e resposta

► Sintaxe das mensagens:

- campos presentes nas mensagens e como são identificados

► Semântica das msgs:

- significado da informação nos campos

► Regras para quando os processos enviam e respondem às mensagens

Protocolos de domínio público:

- definidos em RFCs
- Permitem a interoperação
- ex, HTTP e SMTP

Protocolos proprietários:

- Ex., Skype

De que serviços uma aplicação necessita?

Transferência confiável de dados

- ▶ algumas apls (p.ex. áudio) podem tolerar algumas perdas
- ▶ outras (p.ex., transf. de arquivos, telnet) requerem transferência 100% confiável

Temporização

- ▶ algumas apls (p.ex., telefonia Internet, jogos interativos) requerem baixo retardo para serem “viáveis”

Largura de banda

- ❑ algumas apls (p.ex., multimídia) requerem quantia mínima de banda para serem “viáveis”
- ❑ outras apls (“apls elásticas”) conseguem usar qq quantia de banda disponível

• Segurança

- ❑ Criptografia, integridade dos dados, ...

Requisitos de aplicações de rede selecionadas

Aplicação	Perdas	Largura de Banda	Sensibilidade ao atraso
transferência de arqs	sem perdas	elástica	não
correio	sem perdas	elástica	não
documentos Web	sem perdas	elástica	não
áudio/vídeo em tempo real	tolerante	áudio: 5kb-1Mb vídeo:10kb-5Mb	sim, 100's mseg
áudio/vídeo gravado	tolerante	como anterior	sim, alguns segs
jogos interativos	tolerante	> alguns kbps	sim, 100's mseg
mensagem instantânea	sem perdas	elástica	sim e não

Serviços providos pelos protocolos de transporte da Internet

Serviço TCP:

- ▶ *orientado a conexão:* apresentação requerida entre cliente e servidor
- ▶ *transporte confiável* entre processos remetente e receptor
- ▶ *controle de fluxo:* remetente não vai “afogar” receptor
- ▶ *controle de congestionamento:* estrangular remetente quando a rede estiver carregada
- ▶ *não provê:* garantias temporais ou de banda mínima

Serviço UDP:

- transferência de dados não confiável entre processos remetente e receptor
- não provê: estabelecimento da conexão, confiabilidade, controle de fluxo, controle de congestionamento, garantias temporais ou de banda mínima

P: Qual é o interesse em ter um UDP?

Aplicações Internet: seus protocolos e seus protocolos de transporte

Aplicação	Protocolo da camada de aplicação	Protocolo de transporte usado
correio eletrônico	SMTP [RFC 2821]	TCP
acesso terminal remoto	telnet [RFC 854]	TCP
Web	HTTP [RFC 2616]	TCP
transferência de arquivos	ftp [RFC 959]	TCP
streaming multimídia	HTTP (ex. Youtube) RTP [RFC 1889]	TCP ou UDP
telefonia Internet	SIP, RTP, proprietário (ex., Skype)	tipicamente UDP